第 5-2 讲 • 数据安全：备份、迁移、加密

> **模块 5 • 质量与安全** ｜ 建议用时 **40 分钟**（阅读 22 + 动手 18）
> 前置要求：完成 5-1（理解验证的价值）

本节目标

读完并做完这一讲，你能：

1. 说出记账簿在**四个层次**上保护数据，以及每层防的是什么。
2. 理解**软删除 + 回收站 + 附件保留**这套组合的设计意图（为什么"删了还能救回来"）。
3. 知道**自动备份**的完整策略（多久一次、保留几份、什么时候检查）。
4. 讲清**应用锁**能防什么、**不能**防什么（以及为什么必须诚实说明这一点）。
5. 理解**迁移**是唯一会动用户已有数据的操作，以及它的三道保护。

一、数据安全有四个层次

1.1 先看一张全景图

记账簿保护数据的方式，从"日常小失误"到"灾难恢复"，一共四层：

层	机制	防的场景	用户感觉
① 软删除	删记录不真删，只打标记	**手滑删错一笔**	"回收站里还能找回来"
② 附件保留	彻底删除时默认不删图片	**误删了带收据的记录**	"图片还在，以后还能找到"
③ 自动备份	定期复制整个数据库	**数据被改坏 / 软件出问题**	"退回几天前的状态"
④ 约束与迁移保护	数据库层面的规则 + 升级演练	**代码写错导致数据脏**	"软件不让我做错事"

这四层不是随便堆的，它们的**防护强度递增、恢复代价也递增**：

```
手滑删错      →  点一下"恢复"就好          （代价：1 秒）
图片被删      →  去数据目录手动找          （代价：几分钟）
数据被改坏    →  从备份文件恢复            （代价：损失几天数据）
数据库损坏    →  靠备份 + 手工修复         （代价：最大）
```

好的安全设计就是这样分层的：让最常见的问题有最便宜的解法。

1.2 一个重要认知：没有"绝对安全"

必须先说清楚一件事：

> **记账簿保护的是"操作失误"和"软件错误"，不是"硬件损坏"和"文件被删"。**

什么意思：

场景	记账簿能保护吗
你手滑删了一笔账	✅ 能（回收站）
软件 bug 改坏了数据	✅ 能（备份）
硬盘坏了	❌ **不能**
你手动删了数据目录	❌ **不能**
中了勒索病毒	❌ **不能**（如果备份也在同一块盘）

这是本地软件的天然边界：所有数据都在**你这一台电脑**上。**电脑出事 = 数据出事。**

正确的做法（软件说明书里也写了）：

> **把数据目录放进你平时会备份的地方**（比如自动同步到网盘的文件夹），或者定期把 `backup\` 里的文件拷到别的地方。

这一条非常重要，我在讲义里要强调三遍：本地软件的安全，最终取决于你有没有把数据备份到"这台电脑之外"。

二、第一层：软删除（2-3 讲回顾 + 深化）

2.1 机制

```
-- 删除时：不是 DELETE，而是写一个时间戳
UPDATE records SET deleted_at = <当前时间> WHERE id IN (...)

-- 查询时：只看没被删的（第 85 行的真实代码）
r.deleted_at IS NULL
```

2.2 完整链路

我实测追踪过这个链路（在 5-1 讲）：如果你把 `deleted_at IS NULL` 改成 `IS NOT NULL`，**9 项检查立刻失败**——说明整个列表查询都建立在这个条件上。

用户看到的样子：

```
点删除 → 记录从列表消失 → 右下角弹出「已删除 · 撤销」
                                    ↓ 点撤销
                              记录回来了（因为数据从来没真的消失）
```

2.3 为什么"软删除"比"真删除"好

不是技术上做不到真删除（SQL 一句 `DELETE` 就行），而是**故意不这么做**：

理由	说明
误删可恢复	记账数据是"记下来就忘"的，用户往往几个月后才发现少了一笔
撤销功能的基础	右下角那个「撤销」按钮完全依赖软删除
数据可追溯	以后想做"我删过哪些账"的分析，数据还在

代价：数据库文件会变大（删掉的东西还占空间）。**但对记账软件来说，这点空间远不如"误删能救回来"重要。**

三、第二层：附件保留（"决策"设计）

3.1 一个真实的设计难题

场景：一条记录带了收据图片。用户要**彻底删除**（不是软删除，是从回收站里清掉）。

问题：那张图片要不要一起删？

两个选项都有问题：

选择	后果
自动删图片	万一用户误操作，**图片永久没了**（图片不在数据库里，删了就是删了）
永不删图片	数据目录里堆一堆无主图片，**占空间**

3.2 记账簿的解法：让用户决定，默认保守

看真实代码（`ipc\records.ts`）：

```
ipcMain.handle(IPC.recordsPurge, (_event, ids: string[], deleteAttachments: boolean): Writ
  assertReady()
  return purgeRecords(ids ?? [], Boolean(deleteAttachments))
})
```

注意那个 `deleteAttachments` 参数——它来自**界面上的一个复选框**（你在回收站页见过）：

```
□ 同时删除这 N 个收据文件
  （默认不勾选：图片会继续留在数据目录，需要时还能找回）
```

这个设计的三层考虑：

设计	理由
给用户选择	有人想彻底清干净，有人想留个后路
默认不删	**删除是不可逆的**，默认值应该选"损失更小"的那个
文案说清后果	用户勾之前得知道"不勾会留着图片"

这是一条通用的产品设计原则：

> **不可逆的操作，默认值必须是"更保守"的那个。**

同样的原则你在别处也见过：删除确认弹窗、清空回收站的二次确认。

3.3 `selftest` 里有对应的检查

回顾 5-1 讲看过的检查项名字：

- 「软删除不动物理附件」
- 「彻底删除默认保留附件文件」

这两条把上面这个设计规则"钉"住了。以后有人改删除逻辑时如果把图片一起删了，**检查会失败。**

四、第三层：自动备份

4.1 完整策略（真实配置）

默认值（`shared\constants.ts`）：

```
autoBackup: 'weekly',    // 默认每周
backupKeep: 7,           // 保留 7 份
```

用户可调（设置页）：关闭 / 每天 / 每周，保留 3–30 份。

4.2 关键设计：什么时候检查"该不该备份"

这是本讲最值得学的一个决策。

看真实代码（`backup.service.ts`）：

```
export function autoBackupIfDue(mode: 'off' | 'daily' | 'weekly', keep: number): BackupRes
  if (mode === 'off') return null
  const interval = mode === 'daily' ? DAY_MS : DAY_MS * 7
  const info = backupInfo()
  if (info.lastAt !== null && Date.now() - info.lastAt < interval) return null   // 没到期,
  const result = backupNow()
  ...
}
```

它检查的是"上次备份是什么时候"（靠文件修改时间），**到期才跑。**

而在 `main\index.ts` 里，这个检查挂在"应用启动时"：

```
// 启动时检查自动备份
if (config.initialized && isOpen()) {
  const settings = settingsRepo.getAll()
  autoBackupIfDue(settings.autoBackup ?? 'weekly', settings.backupKeep ?? 7)
}
```

为什么放在"启动"而不是"退出"：

放退出	放启动
崩溃/断电时**退出钩子不一定跑得到**	启动成功说明**上次的数据至少是完好的**
备份可能发生在数据已经坏掉之后	趁数据还好的时候留档

这个决策很聪明，而且是很多软件会做错的地方（习惯性地把清理/备份放在退出）。**记住这个思路**：**关键操作要挂在"必然发生"的时机上，而不是"可能不发生"的时机上。**

4.3 备份之前必须做的一件事

看 `backupNow()` 的真实实现：

```
export function backupNow(): BackupResult {
  const source = dbPath(getDataDir())
  if (!existsSync(source)) return { ok: false, message: '数据库文件不存在' }
  checkpoint()      // ★ 关键：先把内存里的改动落盘
  const target = path.join(dir, `${PREFIX}${timestamp()}${SUFFIX}`)
  copyFileSync(source, target)
  return { ok: true, filePath: target }
}
```

★ `checkpoint()` **是什么**：

看一下它的真实实现（`db\connection.ts`）：

```
export function checkpoint(): void {
  if (!db) return
  try {
    db.pragma('wal_checkpoint(TRUNCATE)')     // ★ 就是这一句
  } catch {
    /* ignore */
  }
}
```

`wal_checkpoint(TRUNCATE)` **在做什么**：

SQLite 为了提高速度，会把改动先写进一个**临时文件**（`daily-ledger.db-wal`），**不是每次都立刻写进主数据库文件。**（这就是 1-2 讲你看到的那个 `daily-ledger.db-wal` 文件。）

所以：如果你直接复制 `daily-ledger.db`，**可能复制到一份"少了最近几笔"的旧数据**——因为最新的改动还躺在 `.db-wal` 里没合并进去。

`wal_checkpoint(TRUNCATE)` **的作用**：**强制把那些暂存的改动全部合并进主文件，并清空 WAL 文件。**在这之后复制，才是完整的。

这是一个非常容易忽略的细节——很多自己写备份功能的人都会踩这个坑。**记住：备份 SQLite 之前必须先 checkpoint。**

顺带一提：`catch { /* ignore */ }` 表示"checkpoint 失败了也别中断备份"。**这是权衡**：宁可复制一份可能不完整的备份，也比完全不备份好。

4.4 自动清理

```
export function pruneBackups(keep: number): number {
  const limit = Math.max(1, Math.floor(keep))
  const files = listBackupFiles()          // 按时间从新到旧排
  let removed = 0
  for (const item of files.slice(limit)) { // 超出 keep 的
    unlinkSync(item.file)                  // 从旧的开始删
    removed += 1
  }
  return removed
}
```

它的行为：**只保留最新的 N 份，更旧的自动删掉。**

为什么必须有它：否则备份会无限堆积，**一年后占满硬盘**——那反而成了新问题。

注意一个细节：`Math.max(1, ...)` 保证**至少留 1 份。即使配置错误也不会把备份全删光。** 这是防御性编程。

五、第四层：应用锁（以及它的诚实边界）

5.1 机制（真实实现）

看 `security.service.ts`：

```
const KEY_LEN = 32

export function hashPin(pin: string): { hash: string; salt: string } {
  const salt = randomBytes(16).toString('hex')          // ① 随机盐
  const hash = scryptSync(pin, salt, KEY_LEN).toString('hex')  // ② 慢哈希
  return { hash, salt }
}

export function verifyPin(pin: string, hash: string, salt: string): boolean {
  const candidate = scryptSync(pin, salt, KEY_LEN)
  const expected = Buffer.from(hash, 'hex')
  if (candidate.length !== expected.length) return false
  return timingSafeEqual(candidate, expected)           // ③ 定时安全比较
}
```

三个安全细节，逐个解释：

细节	为什么这么做
① 随机盐	同一 PIN 每次哈希结果都不同，**防止"查表"破解**（攻击者不能预先算好所有 PIN 的哈希）
② scrypt（慢哈希）	**故意让它慢。我实测过：单次约 24 毫秒**（比普通哈希慢几万倍）。正常用户输一次无所谓，但**暴力尝试一万个 PIN 要 4 分钟**，十万个要 40 分钟
③ 定时安全比较	普通 `===` 会在第一个不同的字符处提前返回，**攻击者能通过耗时推测出正确的字符。**`timingSafeEqual` 固定耗时，**不泄露信息**

关于第 ② 点的实测数据：我跑了一段测试（5 次哈希共 119 毫秒）：

```
5 次耗时 119 ms；单次约 24 ms
```

对比一下：普通哈希（如 MD5）算一次是**微秒级**（0.000001 秒）。scrypt 慢了几万倍——**这正是它的目的。**

这就是"防暴力破解"的原理：不是让人猜不到，而是**让尝试变得太贵。**

这三个细节是"密码存储"的标准做法。 我实测过（`verify:security` 20 项检查）：同一个 PIN 两次哈希的盐不同、错 PIN 必拒、清空后失效。

5.2 但必须诚实：它能防什么，不能防什么

这是本讲最重要的一段。 看项目里真实的说明文案：

> **说明白一点**：PIN 是加盐哈希后存在数据库文件里的，它挡的是「别人随手打开你的电脑」，**不是**「把 `daily-ledger.db` 拷走也读不出」。

为什么挡不住"拷走文件"：

```
你设了 PIN  → 数据库里存的是哈希（安全的）
但数据库本身没有加密 → 里面的账目是明文
有人直接拷走 daily-ledger.db → 用任何 SQLite 工具打开 → 全部账目一览无余
```

类比：**PIN 是给房门装的锁，不是给保险柜装的锁。** 锁了门，但有人把整个柜子搬走，锁就没用了。

要真正做到"拷走也读不出"，必须**整库加密**（业界方案叫 SQLCipher）。**项目目前没做**，因为：

考虑	说明
需要更换数据库组件	从 `better-sqlite3` 换成支持加密的分支
要迁移现有数据	**风险较高**（迁移是唯一会动已有数据的操作，见下节）
需要用户确认	这是"改变数据存储方式"的重大决定

把这条边界写清楚，比含糊地说"我们有应用锁更安全"要诚实得多。

这是本讲真正想教你的东西：

> **做安全功能时，要明确说出它的边界。含糊其辞的安全说明，比没有更危险——因为它让人产生错误的安全感。**

5.3 连错 5 次会冷却

界面上还有个细节（`LockScreen.tsx`）：**连错 5 次，冷却 10 秒。**

为什么：给暴力尝试增加成本。**虽然挡不住有耐心的攻击者，但能挡住"随手乱试"。**

这也是"分层防御"的思路：每一层都不完美，但叠加起来足以应对常见威胁。

六、迁移：唯一会动已有数据的操作

6.1 为什么它最危险

回顾 2-3 讲：迁移是"把旧数据库结构升级到新结构"的代码。

为什么它是全项目最危险的操作：

操作	会动已有数据吗
改界面	❌
改主进程逻辑	❌（只影响新操作的行为）
跑迁移	✅ **会，而且是大范围重写**

迁移出错 = 用户的账就毁了。 所以它有三道保护。

6.2 三道保护（真实代码）

第一道：包在事务里（要么全成功，要么全回滚）

```
const apply = db.transaction(() => {
  migration.up(db)                                  // 执行结构变更
  db.pragma(`user_version = ${migration.version}`) // 更新版本号
})
apply()
```

如果中间任何一步失败，事务整体回滚——数据库回到执行前的状态。**绝不会出现"改了一半"的残缺状态。**

第二道：外键开关的精确控制

```
if (migration.foreignKeysOff) db.pragma('foreign_keys = OFF')
try {
  ...执行迁移...
} finally {
  if (migration.foreignKeysOff) db.pragma('foreign_keys = ON')   // 一定要开回来
}
```

为什么要临时关外键：重建表（SQLite 改表结构的标准做法是"建新表 → 搬数据 → 删旧表 → 改名"）时，中间状态会短暂违反外键约束。

注意 `finally`：无论成功失败，都要把外键开回来。否则后续所有操作都失去外键保护——**这是个隐蔽的灾难。**

第三道：升级演练（`verify:upgrade`）

这是最关键的一道：它**造一个旧版本的数据库，跑真实迁移，检查数据完好。**

我在 2-3 讲给过你实测证据：跑完 `verify:upgrade`（21/21 通过）后，你的数据库**字节数和修改时间都没变**——它在沙箱里跑，用假数据演练真迁移。

这三道保护的思路值得记住：

> **对危险操作，不要只靠"小心"，要靠机制**：事务保证原子性、显式恢复保证状态干净、演练保证上线前验证过。

七、动手任务

步骤 1：确认你的三层保护都开着

```
cd "C:\Users\34123\Desktop\记账簿"

# ① 软删除：看查询条件
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "deleted_at IS NULL"

# ② 自动备份：看默认配置
Select-String -Path src\shared\constants.ts -Pattern "autoBackup|backupKeep"

# ③ 备份目录在哪（看你的真实数据）
Get-ChildItem "<你的数据目录>\backup" -ErrorAction SilentlyContinue | Select-Object Name, @
```

回答三个问题：

- 你的自动备份设置是什么？（默认每周）
- `backup` 目录里有几份备份？
- 最近一份是什么时候的？

如果 `backup` 是空的：说明还没到备份周期，或者你从没让它跑过。**这不影响使用，但建议你手动备份一次。**

步骤 2：手动备份一次，并确认它有效

在记账簿里：设置 → 数据 → 立即备份。

然后检查：

```
Get-ChildItem "<你的数据目录>\backup" | Select-Object Name, @{n='KB';e={[math]::Round($_.Le
(Get-Item "<你的数据目录>\daily-ledger.db").Length
```

预期：backup 目录多了一个文件，**大小和主数据库接近**（因为它是完整副本）。

验证备份是"完整的"（这一步很专业）：

```
# 用项目的验证工具检查备份文件是不是完好的数据库
& node_modules\electron\dist\electron.exe scripts\verify-db.cjs "<你的数据目录>\backup\<刚才
```

预期：完整性校验通过。

为什么要验：**一个坏掉的备份比没有备份更危险**——因为它给你虚假的安全感。**备份完必须验证过至少一次。**

步骤 3：验证"软删除真的能恢复"

这个实验安全（用你自己的一笔记录）：

1. 在记账簿里删掉一笔**不重要**的记录。
2. 去「回收站」页找到它。
3. 点「恢复」。
4. 回记录页确认它回来了。

你已经做过类似的实验（2-3 讲），这次**带着"这是第四层保护"的意识再做一遍。**

步骤 4：读一遍"备份保留"逻辑

```
Select-String -Path src\main\services\backup.service.ts -Pattern "export function pruneBac
```

读懂它之后回答：

- 如果设置成"保留 3 份"，但有 5 份备份，它会删哪两份？
- **为什么有** `Math.max(1, ...)`？

答案：

- 删**最旧的两份**（因为它按时间从新到旧排，`slice(limit)` 取的是超出的部分）
- `Math.max(1, ...)` 保证**至少保留 1 份**，即使配置传了 0 或负数

步骤 5：检查你的"第二份备份"

这一步最重要，也最容易被跳过。

回答这个问题：

> **如果明天你的电脑硬盘坏了，你的账还在吗？**

如果答案是"在" —— 说明你把数据目录同步到了网盘，或者拷到了移动硬盘。**很好。**

如果答案是"不在" —— 那么**记账簿的四层保护都救不了你。**因为：

```
软删除 → 数据还在同一块盘上
备份   → 备份也在同一块盘上（<你的数据目录>\backup）
```

建议做法（任选其一）：

方案	怎么做
放进网盘同步目录	把数据目录改成网盘同步的文件夹（如"我的坚果云\记账"），在设置里改
定期手动拷贝	每月把 `<你的数据目录>\backup\` 里的文件拷到 U 盘或另一块盘
用同步工具	用 Syncthing、FreeFileSync 之类的工具定期同步到别处

这一条我要强调：**这是整个讲义里最实用的一条建议。** 软件能做的都做了，剩下的部分只能靠你。

卡住了怎么办

现象	原因	怎么办
`backup` 目录是空的	还没到备份周期	手动备份一次（步骤 2）
手动备份报错	数据目录不可写	检查 `<你的数据目录>` 的权限
回收站里找不到刚删的	可能被彻底删了	那只能从备份恢复（这就是备份的价值）
`verify-db.cjs` 报错	路径不对	用完整路径，注意引号
不敢动真实数据	**完全可以理解**	**跳过步骤 3、4 的实验**，只读代码也行

八、自测题

1. 记账簿保护数据有哪四个层次？分别防什么？

2. 本地软件的安全边界在哪？记账簿不能**保护什么？**

3. 彻底删除记录时，为什么默认不**删收据图片？**

4. 自动备份为什么放在"应用启动时"检查，而不是"退出时"？

5. 备份数据库之前为什么要先 `checkpoint()`？

答案与解析

1. ① **软删除**（防手滑删错，可恢复）；② **附件保留**（防误删图片，默认不删文件）；③ **自动备份**（防数据被改坏，可退回几天前）；④ **数据库约束与迁移保护**（防代码写错导致数据脏）。 → 层次设计的原则：**让最常见的问题有最便宜的解法**（手滑删错只需点一下恢复）。

2. 它**不能**保护：**硬件损坏、你手动删数据目录、勒索病毒**（如果备份也在同一块盘）。 → 因为所有数据都在**同一台电脑**上。**本地软件的终极安全取决于"你有没有把数据备份到这台电脑之外"。**

3. 因为**删除是不可逆的**，而图片不在数据库里——**删了就真没了。** → 原则：**不可逆的操作，默认值必须是"更保守"的那个。**同时给用户选择（复选框）+ 文案说清后果。

4. 因为**崩溃或断电时，退出钩子不一定跑得到**；而**启动成功说明上次的数据至少是完好的**，这时候留档最稳。 → 通用思路：**关键操作要挂在"必然发生"的时机上，而不是"可能不发生"的时机上。**

5. 因为 SQLite 会把一部分改动暂存在内存或 `-wal` 临时文件里，**不立刻写进主数据库文件。** → 直接复制可能得到一份"少了最近几笔"的旧数据。`checkpoint()` **强制把暂存改动全部落盘**，之后复制才是完整的。**这是自己写备份功能时最容易踩的坑。**

九、术语表

术语	一句话解释
软删除 / Soft Delete	不真删，只标记删除时间
回收站	集中查看与恢复软删除记录的地方
不可逆操作	做了就回不来的操作（删文件、清空回收站）
保守默认值	不可逆操作时的默认选项，选损失更小的那个
checkpoint	把 SQLite 暂存的改动强制写入主文件
WAL	SQLite 的暂存文件机制（`-wal` 后缀）
随机盐 / Salt	哈希前加的随机字符串，防止查表破解
scrypt	一种故意很慢的哈希算法，用于密码存储
定时安全比较	固定耗时的比较，避免通过耗时推测内容
应用锁的边界	只防"随手打开"，不防"拷走文件"
SQLCipher	整库加密方案（本项目未采用）
升级演练	用假数据跑真实迁移，验证安全性

十、本讲与项目的对应

- 你查看的文件：`backup.service.ts`、`security.service.ts`、`migrations\index.ts`
- 你做的操作：手动备份一次、验证备份完整性、软删除并恢复
- **本讲的备份操作是安全的**（备份不会改原数据）｜**软删除实验会动你的真实数据，但完全可恢复**
- 上一讲：5-1 验证脚本在防什么 ｜ 下一讲：**5-3 隐私与离线：这个项目的结构性优势**

讲义版本：v1 ｜ 对应项目版本：0.1.0